

NSL Compiler Pipeline Composition: Stage Mapping, Invariants, and Failure Modes

Status: Research paper (composition paper for the fourteen prior technique papers) **Date:** 2026-04-16 **Audience:** Engineers joining NSL development; reviewers assessing cross-technique integration; implementers extending the pipeline with a fifteenth technique

Abstract

Thirteen technical papers describe individual NSL compiler techniques — CFTP, FASE, CSHA, WRGA, CPDT, CPKD, CNAG, CPFT, CTQS, CFA, FA v2 emitter, @export, PCA, M35 FP8 MMA — plus the fourteenth, CFIP. Each paper is complete as an independent description, but none explains how the techniques compose into a single pipeline, which decisions must be ordered relative to which, which invariants span technique boundaries, or which failure modes arise only in combination. This paper fills that gap. It maps every technique to its pipeline stage, enumerates twenty-three cross-technique invariants, orders the decisions that must resolve before other decisions can run, and catalogs fourteen failure modes where two techniques' contracts could conflict. A canonical compilation walkthrough traces a GPT-style forward-and-backward pass from parse through runtime, showing where each technique contributes. A final section describes the checklist for adding a fifteenth technique.

This is not a new technique paper. It is the institutional knowledge that survives when the people who wrote the first fourteen move on.

§1 Introduction and scope

§1.1 Why a composition paper

The shipped NSL compiler contains approximately 131K lines of Rust across eight crates. Every one of the fourteen technique papers has landed partially or fully into that codebase. The techniques compose — a training run uses CFTP precision, FASE gradient deferral, CSHA fused attention, WRGA adapter fusion, @export for model methods, CFIP interior formats, and M35 FP8 MMA simultaneously. But the compositions themselves are not documented anywhere. They exist only as the shape of the pipeline that emerges when the techniques are run together.

This is the standard institutional-knowledge problem. Individual papers describe "how does technique X work?" The composition paper describes "what breaks if you change X?" The former is what you need to implement a technique. The latter is what you need to modify one without breaking the others.

§1.2 What this paper is not

This paper does not re-derive or re-explain any of the fourteen techniques. It does not propose new techniques. It does not reproduce the implementation details from any of the source papers. Readers are assumed to have read the technique papers they care about; this paper treats each technique as a black box with a contract.

§1.3 Organization

§2 introduces the seven pipeline stages. §3 maps the fourteen techniques onto those stages, one subsection per technique. §4 enumerates the twenty-three cross-technique invariants. §5 gives the decision ordering constraints — which decisions must resolve before which. §6 catalogs the fourteen cross-technique failure modes and their mitigations. §7 is the end-to-end compilation walkthrough. §8 is the checklist for adding a fifteenth technique.

§2 The pipeline stages

The NSL compiler has seven stages. They run in strict order. Most techniques live in exactly one stage; several live across stages with explicit hand-off contracts. These stages are:

Stage 1 — Parse. Lexical analysis and syntactic parsing produce the AST. AST node types are defined in `(nsl-ast)`. This stage introduces no technique decisions; it only records what the user wrote. Shape dimensions are parsed as expressions and left unresolved.

Stage 2 — Semantic. Name resolution, type checking, shape inference, trait resolution, effect inference, and datatype declaration processing. The key artifact is a semantically-typed AST (STAST) in which every expression has a known type including tensor shapes, dtypes, and effect sets. This stage enforces shape compatibility, catches type mismatches, and records the declarative structure that subsequent stages read.

Stage 3 — Effect/trait/quantization pre-planning. Annotations (`@grad`, `@quant`, `@checkpoint`, `@fp8_compute`, `@fuse_graph`, `@shard`, `@real_time`, `@perf_budget`, `@export`) are resolved against the typed AST. Each annotation transforms the AST in a well-defined way — for example, `@grad` produces a forward/backward pair; `@shard` attaches sharding metadata. The key artifact is an annotated STAST where every operation knows its precision, fusion, sharding, and export policy.

Stage 4 — WGGO (the global planner). The whole-graph optimizer reads the annotated STAST and produces per-layer decisions. For each layer it decides whether to prune, which CSHA level to use, which adapter rank (if WRGA), whether to apply FASE Deferred, and which CFIP interior formats to assign. WGGO's output is a decision table: one row per layer, one column per per-layer decision. This decision table is read by every downstream stage. WGGO is the single most important stage for cross-technique coordination — it is where techniques see each other.

Stage 5 — IR lowering. The annotated STAST plus WGGO's decision table is lowered to NSL's middle-end IR (MIR), which contains explicit tensor operations, memory placements, and kernel boundaries. This is where the abstract tensor-dataflow representation becomes an executable plan. Kernel boundaries are fixed by this stage; CFIP's interior-tensor analysis runs against a fully-committed fusion plan.

Stage 6 — Codegen and fusion. MIR is lowered to Cranelift IR for CPU paths and to custom PTX emitters for GPU paths. CFA (CUDA Fused Attention scalar emitter, FA v2) generates attention PTX. CSHA generates the fused-attention epilogue variants. WRGA generates the adapter-fused PTX. M35 generates FP8 MMA instructions. CFIP selects the interior format for each kernel-interior tensor and emits the appropriate lane-local computation.

Stage 7 — Runtime. The compiled binary is executed. The NSL runtime handles tensor allocation, kernel dispatch, distributed communication (CPDT), gradient accumulation (FASE), optimizer steps, and serving-loop coordination. Runtime-visible policies (FASE's AdamW bias correction, CPDT's all-reduce, @export's C-ABI boundary) live here.

These seven stages are not negotiable. Techniques that need to exist in two stages declare an explicit contract between their Stage-N and Stage-M components. Techniques that attempt to reach across more than one stage without an explicit contract are bugs waiting to happen — this has happened twice in project history, both caught in review.

§3 Technique-to-stage mapping

This section describes where each of the fourteen techniques lives in the pipeline and what contracts it exposes to neighbors. The format is: **stages it operates in, what it produces, what it reads, what it assumes.**

§3.1 CFTP — Tensor-Core Fused Precision (FP8 + 4-bit residual)

Stages: 6 (codegen, primarily), 7 (runtime dequant paths).

What it produces: Per-weight-tensor FP8 + 4-bit residual encoding during kernel emission. The FP8 tile and the 4-bit residual tile share a scale factor stored once per tile. The emitted PTX performs the recomposition $W_{full} = W_{fp8} + s * W_{4bit}$ before the MMA.

What it reads: WGGO's decision on whether a given layer uses CFTP (vs dense FP16, vs CTQS integer quantization, vs external NSL CFIP interior). The per-tensor scale factors arrive from a calibration pass that runs before codegen.

What it assumes: The target GPU has tensor cores supporting the chosen FP8 format (M35 variant, typically E4M3 for weights and E5M2 for activations). Tile sizes are divisible by the MMA shapes M35 declares. The weight tensors fit the FP8 ± 4-bit residual format's dynamic range, which is checked at calibration.

Downstream readers: M35 consumes CFTP's FP8 output format. CFIP reads CFTP's "this layer is CFTP" flag and treats the interior of CFTP-using kernels as potentially compatible with CFIP interior formats (they compose; §4 addresses this).

§3.2 FASE — Flash Activation Streaming Epochs (deferred gradient accumulation)

Stages: 3 (to insert the per-layer accumulation annotation), 7 (runtime execution of the Deferred / Immediate / Off modes).

What it produces: A per-layer decision: Deferred (no gradient buffer, accumulate into optimizer state directly), Immediate (standard gradient buffer), or Off (inference-only). Runtime code that reads this decision and dispatches the right accumulation path.

What it reads: @grad annotations, WGGO's per-layer memory pressure estimate, the compile-time known batch size and accumulation steps.

What it assumes: The optimizer state is present in memory at gradient-application time. The AdamW bias correction path uses `ns1_bias_correction_inv` on `MHat`/`VHat` registers with `ScalarMulByBc` — this is an FFI-level invariant that both FASE's runtime and the optimizer implementation must respect (see §4 I-03).

Downstream readers: CFIP's memory analysis reads FASE's Deferred decisions to know how much memory the gradient buffer is *not* occupying. CPDT's all-reduce scheduling reads FASE's Deferred flag to know gradients are directly in optimizer state (not in a separate buffer to be reduced).

§3.3 CSHA — Compiler-Synthesized Hardware Attention

Stages: 4 (per-layer tier selection in WGGO), 5 (fusion planning), 6 (PTX emission of tier-specific kernels).

What it produces: Per-attention-layer decision: Tier A (fully-fused forward and backward with recompute), Tier B (fused forward with explicit backward), Tier C (fused forward with save-activations for a separately-generated backward). The chosen tier determines what the forward kernel writes to HBM (boundary contract) and what the backward kernel expects in HBM.

What it reads: Attention layer shape (`num_heads`, `head_dim`, `seq_len`), GQA/MQA grouping from @gqa, RoPE frequency from @rope, the presence of masking (causal, segment, padding).

What it assumes: `head_dim` $\leq$ 256 (FA v2 emitter constraint). Sequence length at compile time is either known (static) or within a declared `seq_range` bound. The backward kernel exists and matches the forward's save-activation contract byte-for-byte.

Downstream readers: CFIP reads the Tier decision to determine which attention-interior tensors are available for interior-format assignment (Tier A has many; Tier C has few because save-activations writes several to HBM). PCA reads the Tier decision to know whether segment masking can be fused into the forward kernel or must be applied separately.

§3.4 WRGA — Wengert-Pruned Roofline-Guided Adaptation

Stages: 4 (per-adapter decision in WGGO), 5 (fusion plan), 6 (PTX emission of the fused LoRA/IA³ kernel).

What it produces: Fused PTX for LoRA, IA³, or gated variants. The fused kernel performs the base MMA and the adapter MMA in one SM-resident pass, with the adapter's rank-r computation held in register fragments across the K-loop boundary. The LoRA epilogue reuses the A-fragment of the main MMA — this is a load-bearing optimization because the m16n8k16 A-fragment is B-matrix-independent.

What it reads: Adapter type (LoRA/IA³/gated), rank, base matrix shape, and which dimensions are adapted.

What it assumes: The adapter rank is small enough that the adapter MMA fits in the register file after the main MMA's staging. Sub-MMA-shape tiles zero-pad in SMEM (option 1 in the design doc). ptxas validation runs at build time and rejects emitter regressions.

Downstream readers: CFIP reads WRGA's adapter fusion to know that the adapter's low-rank intermediate tensor is kernel-interior and can use CFIP formats (§4 I-09). CSHA Tier C save-activations is orthogonal — WRGA kernels do not write attention activations.

§3.5 CPDT — Compiler-Planned Distributed Training

Stages: 3 (read @shard annotations, tensor parallelism, context parallelism, pipeline parallelism), 4 (sharding decisions fold into WGGO's memory model), 7 (runtime all-reduce, all-gather, reduce-scatter, pipeline scheduling).

What it produces: A distributed execution plan: which tensor is sharded on which dimension across which process group, which all-reduces are required at which layer boundaries, which pipeline stages contain which layers, which activations must be recomputed on the receiving rank.

What it reads: @shard annotations (tp, cp, pp, dp), the process topology at runtime, the model's layer structure, the presence of sequence-parallel regions.

What it assumes: The process group topology is known at runtime initialization and does not change during training. All-reduce is mathematically equivalent to local gradient sum followed by broadcast — this assumption breaks if gradient compression is introduced (see §8 gap analysis).

Downstream readers: FASE's Deferred mode must publish to CPDT that there is no separate gradient buffer to reduce — the all-reduce must reduce the *optimizer state gradient accumulator*, not the gradient buffer. CFIP's memory analysis reads CPDT's sharding decisions to know how large each rank's kernel-interior tensors are.

§3.6 CPKD — Compiler-Packed Knowledge Distillation

Stages: 3 (resolve @distill annotation), 4 (WGGO schedules teacher-student forward passes), 5 (fusion of teacher and student forward kernels where profitable).

What it produces: A per-training-step distillation schedule. Teacher forward is runtime-cached (same weights across steps), student forward runs fresh, the distillation loss is computed from both logits. When teacher and student share kernel shapes, CPKD fuses their forward passes into shared-launch kernels.

What it reads: @distill annotations on the student model, the teacher model's compile artifact (typically a separate NSL-compiled binary loaded via @export).

What it assumes: Teacher weights are frozen. Teacher and student share the same tokenizer (otherwise the logit-level loss is ill-defined). The teacher model's @export boundary returns logits before softmax (matching the student's logit output).

Downstream readers: @export's CPKD-targeted variants must produce logits, not post-softmax probabilities. The grad-context bridge must not flow gradients into the teacher (§4 I-11).

§3.7 CNAG — Compiler-Native Adaptive Gradient

Stages: 3 (read @adaptive on optimizer declarations), 7 (runtime adaptive step-size logic).

What it produces: A runtime adaptive step-size routine that adjusts the learning rate per-layer based on gradient norm, loss trajectory, and parameter norm. The routine is specialized at compile time to the optimizer type (AdamW, SGD, Lion, Shampoo).

What it reads: @adaptive annotations, the optimizer state layout, the loss value from the current step.

What it assumes: The loss value is available to the optimizer step (this is an obvious-seeming assumption that breaks in pipeline-parallel settings where the loss is computed on the last pipeline stage — CPDT must route it back).

Downstream readers: FASE's Deferred mode must publish the accumulated gradient norm through a statistics hook that CNAG reads. This is a runtime FFI contract

(`ns1_grad_norm_read`) rather than a compile-time one.

§3.8 CPFT — Compiler-Planned Fine-Tuning

Stages: 4 (WGGO decides fine-tune-specific policies), 7 (runtime freeze/unfreeze scheduling).

What it produces: Per-layer fine-tuning policies: fully-frozen (no gradient flows), adapter-only (only LoRA/IA³ gradients flow), partially-frozen (certain parameter groups frozen), trainable. The policy determines which gradient computations are emitted and which are skipped.

What it reads: @finetune annotations, the set of WRGA adapters registered, the base-model compile artifact.

What it assumes: The base model's weight layout is stable between fine-tune runs (@export's weight-ptr ABI enforces this). Frozen layers' activations are still needed for forward propagation even when no gradient flows back — this is subtle and has tripped implementers.

Downstream readers: WRGA receives CPFT's "this layer is adapter-only" decision and enables its fused-LoRA path. FASE may skip gradient buffer allocation for frozen layers entirely (no gradient flows → no buffer needed).

§3.9 CTQS — Compiler-Trained Quantization Schedules

Stages: 3 (read @quant_schedule), 4 (WGGO fold schedule into per-layer quant decisions), 7 (runtime schedule-following quantization).

What it produces: A training-time schedule that varies quantization aggressiveness across training phases — typically starting from full precision, progressively introducing FP8, then 4-bit residual, then aggressive CFIP interior formats. Each phase transition is triggered by training step count, loss plateau detection, or explicit schedule.

What it reads: `@quant_schedule` annotations (phase boundaries, per-phase format assignments), training step count from the trainer loop.

What it assumes: The loss does not catastrophically degrade at any phase transition — this is calibrated at schedule design time and checked at runtime with a loss guard that can roll back a phase transition if loss exceeds a threshold.

Downstream readers: CFTP's per-layer decision reads CTQS's current phase. CFIP's format assignment reads CTQS to know whether aggressive interior formats are yet eligible.

§3.10 CFA — CUDA Fused Attention (FA v2 scalar emitter)

Stages: 6 (PTX emission for attention kernels).

What it produces: PTX for flash-attention-style forward and backward kernels. The emitter is shape-parametric over (`num_heads`, `head_dim`, `seq_len`, `tile_M`, `tile_N`). It handles causal masking, RoPE, GQA, and the softmax's online-softmax normalization.

What it reads: CSHA's chosen tier, shape parameters, masking specification, dtype specification (FP16/BF16/FP8 via M35).

What it assumes: `head_dim` $\in \{64, 96, 128, 192, 256\}$, with `head_dim` = 128 being the well-tested primary path. Smaller/larger `head_dim` values take less-optimized code paths and are subject to the `head_dim` diagnosis currently pending on Track B.

Downstream readers: CSHA's tier dispatch reads CFA's output. PCA's segment-masked variant wraps CFA's core with a segment-ID-checking softmax prologue.

§3.11 `@export` — Model Method Export

Stages: 3 (resolve `@export` annotations), 5 (generate the C-ABI wrapper), 7 (FFI dispatch at runtime).

What it produces: A C-ABI function per `@export`-annotated model method. The function signature is `((weight_ptrs, num_weights, inputs..., outputs...))`. Internally, the function unpacks weight pointers into the model's weight registry, runs the declared method, and returns outputs.

What it reads: The `@export` annotation, the method's NSL signature (which becomes the C-ABI input/output layout), the model's weight layout (from the base model compile).

What it assumes: Weight ordering at `@export` time matches weight ordering at dynamic-load time. Inputs and outputs are f16/f32 at the boundary (CFIP interior formats are invisible across the boundary — see §3.14). The caller owns the input/output buffers.

Downstream readers: External Python callers via ctypes; CPKD's teacher-model loading; any external serving infrastructure that wants to call NSL-compiled methods.

§3.12 PCA — Per-Cross-Attention segment-masked attention

Stages: 4 (WGGO identifies PCA-eligible attention), 6 (segment-aware PTX emission).

What it produces: A PCA-variant attention kernel that applies a segment-ID mask during the softmax. The mask zeros out K-Q pairs whose segment IDs differ, preventing cross-segment attention. The `segment_ids` tensor is a parallel input alongside Q, K, V.

What it reads: `@segment_masked` annotation, the `segment_ids` tensor type (must be integer), the attention shape.

What it assumes: `segment_ids` is contiguous within segments (i.e., within a batch, segment 0 tokens come before segment 1 tokens, etc.). Segment boundaries align with powers-of-2 in tile size for the efficient masking path — otherwise a slower generic path is used.

Downstream readers: Segment-boundary information is known to PCA but *not currently exposed* to CSHA's tile scheduler (see the connections paper for the proposed integration).

§3.13 M35 — FP8 MMA instruction emitter

Stages: 6 (PTX emission).

What it produces: The FP8 variant PTX instructions

(`(mma.sync.aligned.m16n8k32.row.col.f32.e4m3.e4m3.f32)`, etc.). Accurately models the accumulator type (f32), the input types (e4m3 or e5m2), and the scale-factor propagation.

What it reads: The per-tile FP8 format decision (from CFTP or CTQS).

What it assumes: SM_89 or later (Ada, Hopper, Blackwell). The GPU actually supports the requested FP8 format — checked at compile time against `--gpu-arch`.

Downstream readers: CFTP, CTQS, CFIP (when interior formats include FP8 variants), and WRGA's LoRA-with-FP8-weights path.

§3.14 CFIP — Compiler-Fused Interior Precision

Stages: 5 (kernel-boundary analysis), 6 (interior-format assignment during PTX emission).

What it produces: Per-interior-tensor format assignments (IF4, IL2, IM3, or unchanged f16/f32). Interior tensors are those that never leave a fused kernel; their format affects only SMEM and register usage, not HBM. The format assignment respects the Gradient-Envelope Bit Allocation (GEBA) bound.

What it reads: The fused-kernel boundary from MIR (finalized at Stage 5), the tensor role within each kernel (from MIR metadata), WGGO's per-layer sensitivity marker, the GEBA symbolic bound.

What it assumes: Kernel fusion boundaries are stable from Stage 5 onward. The GEBA bound is tight enough that the symbolic bit budget is not dominated by slack.

Downstream readers: None — CFIP is the last technique in the pipeline. Its output directly shapes the emitted PTX.

§4 Cross-technique invariants

The following twenty-three invariants must hold for the compiled binary to be correct. Each has a source technique (the one that imposes the invariant) and a consumer technique (the one that must respect it). Invariants are numbered I-01 through I-23; numbering is stable for cross-reference from future papers.

I-01 — Weight-ptr ordering is stable across @export boundary. *Source:* @export. *Consumer:* any caller, CPKD teacher loading. The weight pointer array ordering is determined by a deterministic traversal of the model's parameter registry. The same model compiled on two different machines must produce the same ordering. Violating this breaks dynamic weight loading.

I-02 — The backward kernel's save-activation contract matches the forward's byte layout. *Source:* CSHA (forward). *Consumer:* CSHA (backward), CFA. CSHA Tier C writes Q_proj/K_proj/V_proj/row_max/row_sum with a specific stride and dtype. The backward kernel reads them with the same stride and dtype. Mismatches cause silent miscompilation (backward reads garbage).

I-03 — FASE Deferred mode preserves AdamW bias correction equivalence. *Source:* FASE. *Consumer:* AdamW optimizer, CNAG. The sequence MHat/VHat with `ScalarMulByBc` and `ns1_bias_correction_inv` produces numerically identical updates to the standard AdamW formulation for any gradient accumulation step count N. The Jensen-fence test enforces this. Violations show up as slow convergence divergence over thousands of steps, not immediate error.

I-04 — Gradient flow follows the grad-context bridge ABI. *Source:* grad-context bridge. *Consumer:* @export, CPDT, @checkpoint. Input gradients are returned by `ns1_model_backward` via the grad-context; they are not written to caller-owned buffers. The caller reads them through `ns1_model_input_grad_read`. Direct buffer writes break the ABI and create aliasing bugs.

I-05 — CFIP interior formats never leave the kernel. *Source:* CFIP. *Consumer:* any kernel that fuses with CFIP-interior-using kernels. An IF4/IL2/IM3-formatted tensor that leaves its originating kernel is a bug. The compiler emits a dequant-on-exit for any boundary tensor that was computed in an interior format; if the dequant is missing, the consuming kernel reads corrupted data.

I-06 — FP8 MMA accumulates in f32 only. *Source:* M35. *Consumer:* CFTP, CTQS, WRGA (FP8 paths), CFIP (FP8 interior). No hardware path accumulates in FP8 or FP16. Accumulation is always f32. Attempting to emit an FP8-accumulating MMA produces invalid PTX.

I-07 — CFTP FP8 + 4-bit residual uses one scale per tile. *Source:* CFTP. *Consumer:* M35 (for tile size alignment), CTQS (for schedule transitions). The scale factor is not per-element; it is per-tile. Tile boundaries are fixed by CFTP's calibration and must match M35's MMA tile shape.

I-08 — WRGA adapter rank $\leq$ register-file residency budget. *Source:* WRGA. *Consumer:* register allocator, ptxas. The adapter MMA must fit in registers after the main MMA's staging. Exceeding this forces spills, which destroy WRGA's performance benefit. The WGGO roofline analysis enforces the rank bound at plan time.

I-09 — Adapter-intermediate tensors are eligible for CFIP interior formats. *Source:* WRGA. *Consumer:* CFIP. The low-rank intermediate produced by LoRA's first matmul (before the second matmul that projects back to the full rank) never leaves the kernel. CFIP can assign an interior format to it. This is an explicit composition hook.

I-10 — PCA segment boundaries are known at compile time if segment_ids is constant-folded; otherwise they are runtime. *Source:* PCA. *Consumer:* CSHA tile scheduler (not currently). In practice, segment_ids is rarely constant-foldable; most uses are per-batch runtime. This invariant is relevant mainly when the model has a fixed context-of-contexts structure.

I-11 — Teacher gradients do not flow into CPKD's student backward. *Source:* CPKD. *Consumer:* grad-context bridge. The teacher model's @export boundary must produce a no-grad tensor. Flowing gradients into frozen teacher weights is not just a correctness issue; it allocates memory for teacher gradient buffers and produces useless computation.

I-12 — CTQS phase transitions don't bypass CFIP's GEBA bound. *Source:* CTQS. *Consumer:* CFIP. When CTQS moves to a more-aggressive phase, it shifts CFTP's bits. CFIP's GEBA bound is recomputed for the new phase; the bound remains binding. Ignoring this creates a moving-target problem where CFIP assigns formats valid in phase N but not in phase N+1.

I-13 — WGGO's decision table is monotonic after Stage 4. *Source:* WGGO. *Consumer:* all downstream stages. Once WGGO commits, the decision table is frozen. Techniques that need to change decisions must loop WGGO — there is no in-place mutation. This is a strong invariant that simplifies reasoning about every downstream stage.

I-14 — CPDT process-group topology is read-only after runtime init. *Source:* CPDT. *Consumer:* FASE, CNAG, any runtime-scheduled technique. Sharding, communication, and reduction all assume fixed process groups. Dynamic topology changes (elastic training) are not supported; adding them requires revisiting every runtime consumer.

I-15 — CFA head_dim $\in \{64, 96, 128, 192, 256\}$. *Source:* CFA. *Consumer:* CSHA, PCA. Other head dims go through a generic slow path that does not exercise the tensor cores. The head_dim = 128 path is the best-tested; 192 and 256 are well-tested; 64 and 96 are the current Track B investigation surface.

I-16 — @export boundary dtypes are FP16 or FP32. *Source:* @export. *Consumer:* any external caller. Interior FP8 and interior CFIP formats are not exposed across the boundary. The wrapper inserts dequant-on-exit and quant-on-enter as needed. Passing FP8 across the @export boundary is unsupported and will be rejected at compile time.

I-17 — CPDT all-reduce reduces the gradient accumulator, not the gradient buffer, when FASE is Deferred. *Source:* FASE. *Consumer:* CPDT. The buffer the all-reduce reduces changes based on FASE's mode. Deferred → reduce MHat/VHat's gradient contribution. Immediate → reduce the gradient buffer. Off → no all-reduce. CPDT's plan reads FASE's mode to pick the right target.

I-18 — Optimizer state is present at gradient-application time. *Source:* FASE, AdamW, CNAG. *Consumer:* runtime. Memory planning must ensure the optimizer state persists across the optimizer step. M36's memory planner enforces this; freeing optimizer state early is a critical bug.

I-19 — Frozen-layer activations are still produced in forward. *Source:* CPFT. *Consumer:* FASE, CSHA, WGGO memory model. A frozen layer's gradient is not computed, but its output is still needed for the next layer's forward. Freezing a layer does not save activation memory unless the downstream consumer is also frozen *and* activation checkpointing is in effect.

I-20 — Datatype declarations are resolved before WGGO. *Source:* BYOD datatype keyword. *Consumer:* WGGO, CFIP. User-defined datatypes (e.g., `datatype Ternary`) must be lowered to a format WGGO and CFIP can reason about before WGGO runs. Deferring datatype resolution to codegen breaks WGGO's decision table.

I-21 — @real_time and @perf_budget annotations flow into WGGO's decision priorities. *Source:* @real_time, @perf_budget. *Consumer:* WGGO. A @real_time(max_latency_ms=5) annotation causes WGGO to favor precision-reducing techniques (CFIP, CFTP) and to reject decisions that would violate the budget. This is not enforced at codegen — it must be visible to WGGO at plan time.

I-22 — RoPE rotation is applied before attention, never within. *Source:* @rope. *Consumer:* CFA, CSHA. The rotation is a pre-multiplication on Q and K; it does not happen inside the attention kernel. Fusing RoPE inside the attention kernel is not supported by CFA; fusing it as a preceding step that writes to Q and K's storage is supported.

I-23 — Sharding boundaries cannot cross fused-kernel boundaries. *Source:* CPDT. *Consumer:* CSHA, CFA, WRGA, CFIP. A tensor that is sharded across processes cannot be a kernel-interior tensor of a fused kernel that spans the shard boundary. This is enforced at Stage 4 (WGGO) by the sharding-compatible-fusion check. Violations would require inter-process communication inside a kernel, which NSL does not support.

§5 Decision ordering constraints

Decisions must resolve in a specific order. Any implementation that reorders them produces an invalid or inconsistent compile. The ordering constraints below are *strict* — they define a partial order on decisions.

O-01 — Datatypes before types. `datatype` declarations must resolve before type checking. The semantic stage processes datatype declarations first, then types.

O-02 — Annotations before WGGO. All @-annotations must be resolved to their semantic effects before WGGO reads the annotated AST. Partial annotation resolution in WGGO is not supported.

O-03 — CPFT freezing decisions before WGGO. Whether a layer is trainable, frozen, or adapter-only must be known before WGGO decides memory layout — freezing saves gradient buffer memory.

O-04 — WGGO before MIR lowering. The entire decision table must be finalized before MIR lowering begins. MIR lowering is read-only with respect to WGGO's decisions.

O-05 — Fusion boundaries before CFIP. CFIP's analysis runs against a stable fusion plan. Attempting to change a kernel boundary after CFIP runs requires re-running CFIP — so, in practice, no such changes happen.

O-06 — CTQS phase before CFIP assignment, per phase. CFIP is aware of the current CTQS phase when selecting interior formats. A new phase triggers re-running CFIP's analysis for that phase.

O-07 — Sharding decisions before fusion boundaries. CPDT's sharding constrains which kernel fusions are legal. Fusion decisions in WGGO are sharding-aware.

O-08 — CPFT freezing before adapter (WRGA) decisions. If a layer is fully frozen (not adapter-only), WRGA does not run on it. If adapter-only, WRGA runs; otherwise WRGA may or may not run (WGGO decides).

O-09 — CPDT topology known before any runtime decision. At runtime initialization, the process group topology is discovered; all runtime decisions read it as constant.

O-10 — Scale-factor calibration before CFTP codegen. The per-tile FP8 scales must be calibrated from data before CFTP emits PTX. This is a compile-time step that runs after WGGO and before codegen.

O-11 — CFA emitter parameters fixed before PTX generation. CSHA's tier, head_dim, tile_M, tile_N, masking, and RoPE presence are all inputs to CFA. They must be fixed at CSHA-tier-selection time and immutable thereafter.

O-12 — @export C-ABI signature fixed at compile time. The wrapper's signature is generated once and not altered. Downstream callers bind to the signature.

These twelve ordering constraints are sufficient to determine a valid pipeline execution order. They do not fully serialize the pipeline — some decisions are independent and can run concurrently.

§6 Cross-technique failure modes

The following fourteen failure modes arise only when multiple techniques compose. Each has a symptom (what the implementer sees), a root cause, and a mitigation.

F-01 — AdamW slow divergence with FASE Deferred. Symptom: training loss slowly drifts from the expected curve over thousands of steps. Root cause: the bias correction path is not identical between Immediate and Deferred modes (violation of I-03). Mitigation: the Jensen fence test in the FASE test suite.

F-02 — Save-activation byte mismatch in CSHA Tier C. Symptom: backward produces NaNs after a few steps. Root cause: the forward and backward kernels disagree on save-activation layout (violation of I-02). Mitigation: a checksum-based test that verifies the saved-activation buffer bit-for-bit between forward and backward.

F-03 — CFIP interior format leakage. Symptom: downstream kernel reads garbage from the boundary tensor. Root cause: missing dequant-on-exit (violation of I-05). Mitigation: the boundary analysis pass validates that every tensor crossing a kernel boundary has a legal dtype; assertion failures at compile time catch the bug.

F-04 — ptxas regression from WRGA emitter. Symptom: generated PTX is rejected by ptxas or produces incorrect results after a WRGA code change. Root cause: the emitter change broke a register allocation assumption. Mitigation: ptxas validation is a mandatory step in the emitter's test suite; the 2026-04-16 WRGA retrospective establishes this as a pattern for all future emitters.

F-05 — FASE + CPDT gradient target mismatch. Symptom: all-reduce fails or produces wrong results. Root cause: CPDT reduces the gradient buffer but FASE is Deferred (no gradient buffer) (violation of I-17). Mitigation: CPDT's plan-time check reads FASE's mode.

F-06 — Teacher gradient accumulation in CPKD. Symptom: memory blow-up during training, teacher parameters slowly drifting. Root cause: grad context flows gradients into teacher (violation of I-11). Mitigation: @export-boundary for teacher models marks outputs as no-grad; the grad-context bridge respects this.

F-07 — head_dim 64/96 performance cliff. Symptom: flash attention runs 3-5x slower on head_dim = 64 than head_dim = 128. Root cause: CFA's 64/96 paths are not tensor-core-optimized (known limitation; Track B). Mitigation: diagnosis in progress; workaround is to use head_dim = 128 when possible.

F-08 — CFIP format assignment inconsistency across CTQS phases. Symptom: training metrics fine in phase 1, degrade at phase-1→phase-2 transition. Root cause: CFIP's phase-1 format assignment not re-derived for phase 2 (violation of I-12). Mitigation: CFIP's phase-transition hook triggers reassignment.

F-09 — WGGO decision table mutation. Symptom: downstream techniques see inconsistent decisions; hard-to-reproduce bugs. Root cause: a pass mutates the decision table after Stage 4 (violation of I-13). Mitigation: the table is frozen (Rust immutable reference) after WGGO completes.

F-10 — Sharding-fusion conflict. Symptom: all-reduce happens mid-kernel; undefined behavior. Root cause: a sharded tensor is a fused-kernel-interior tensor (violation of I-23). Mitigation: sharding-aware fusion check in WGGO.

F-11 — BYOD datatype not visible to CFIP. Symptom: user-defined datatype ignored; compiler uses f16 instead. Root cause: datatype resolution after WGGO (violation of I-20). Mitigation: O-01 plus integration tests that exercise a user-defined datatype under CFIP.

F-12 — CPFT freezing pessimizing activation memory. Symptom: freezing layers does not reduce training memory by the expected amount. Root cause: frozen layer activations are still produced (I-19); the expected memory saving is gradient-buffer savings, not activation savings. Mitigation: documentation. @checkpoint is the mechanism for activation savings.

F-13 — @export dtype rejection. Symptom: @export at compile time rejects a method signature. Root cause: method uses interior dtypes at the boundary (violation of I-16). Mitigation: compiler produces a clear error suggesting FP16/FP32 boundary dtypes.

F-14 — Real-time budget violation due to CFIP re-planning. Symptom: a @real_time(max_latency_ms=5) method exceeds its budget after a CFIP format change. Root cause: the budget check did not re-run after CFIP's decisions (violation of I-21). Mitigation: WGGO's budget check runs after CFIP's format assignment, not just after its own planning.

Each of these failure modes has been encountered at least once during NSL's implementation; many are caught by the existing test suite, but several (F-04, F-11, F-14) arose specifically because an implementer did not have this invariant list in front of them.

§7 End-to-end compilation walkthrough

This section traces a canonical example — a single forward and backward pass through a 6-layer transformer with 8 attention heads, head_dim = 128, d_model = 1024, using FP8 + 4-bit residual weights, FASE Deferred gradient accumulation, a LoRA rank-8 adapter on the output projection, CSHA Tier B attention, CFIP interior formats where applicable, and no distributed training. This walkthrough illustrates where each technique contributes.

Stage 1 — Parse. The user wrote:

```
@grad @fp8_compute
model Transformer(seq_len: 512, d_model: 1024, num_heads: 8, num_layers: 6)
  @flash_attention @rope
  layers: [TransformerBlock; 6]
  @lora(rank=8)
  output_proj: Linear<[d_model, vocab_size]>
```

Parse produces an AST with `TransformerBlock[6]`, attention annotated `@flash_attention @rope`, output projection annotated `@lora(rank=8)`, and the whole model annotated `@grad @fp8_compute`. No technique decisions occur here.

Stage 2 — Semantic. Shape inference verifies tensor shapes are consistent. Effect inference attaches a Grad effect to every operation downstream of `@grad`. Trait resolution binds `@flash_attention` to its implementation (CFA). Datatype declarations (none in this example) would be resolved here.

Stage 3 — Annotations resolve.

- `@fp8_compute` → CFTP is enabled per layer.
- `@flash_attention @rope` → CSHA and RoPE are configured.
- `@lora(rank=8)` → WRGA is activated on the output projection with rank 8.
- `@grad` → the forward AST is paired with a backward AST; grad-context is threaded.

Annotated STAST is frozen.

Stage 4 — WGGO. The global planner reads the annotated STAST. It decides:

- Per layer, CSHA Tier B (fused forward + explicit backward; Tier A would be ideal but Tier C save-activations is not needed in this non-distributed case, and Tier B has the best overall throughput for this size).
- Per layer, CFTP applies to linear projections (Q, K, V, O_proj, MLP up/down). Per-tile calibrated scales.
- For the output projection, WRGA produces a fused LoRA kernel.
- FASE mode: Deferred (the batch size is large enough that deferring the gradient buffer saves ~20% memory).
- Per layer, CFIP interior-format assignments:
 - Attention: row_max, row_sum → IL2 (reduction); attention_weights pre-softmax → IM3 (linear in x); v_accum → IL2 (accumulator) → promoted to f32 at kernel boundary.
 - MLP: up_proj output → IM3; down_proj accumulator → IL2; activation output → IF4 (this is the input to the next attention).
- No sharding (CPDT not active).
- No distillation, no adaptive gradient, no fine-tune freezing, no quantization schedule, no segment masking.

WGGO outputs a decision table with 6 layer-rows × ~15 column-decisions.

Stage 5 — MIR lowering. MIR is produced with explicit kernels:

- Attention kernel (per layer) with inputs Q, K, V, outputs attention_out.
- Output-projection kernel with inputs attention_out, weight, output_proj_lora_A, output_proj_lora_B, outputs logits.
- MLP kernel (per layer) with inputs attention_out, up_proj weight, down_proj weight, outputs mlp_out.

Fusion boundaries are finalized. CFIP's interior-tensor analysis confirms that row_max, row_sum, attention_weights, v_accum are all kernel-interior for the attention kernel; up_proj output and down_proj accumulator are MLP-interior; the LoRA-intermediate is output-projection-kernel-interior.

Stage 6 — Codegen and PTX emission.

- CFA emits the attention PTX, with CSHA Tier B, head_dim = 128, RoPE, CFIP interior formats as specified above, using M35 FP8 MMA instructions for the $Q @ K^T$ and attention_weights @ V.
- WRGA emits the fused output-projection PTX, reusing the A-fragment of the main MMA for the LoRA epilogue. CFIP assigns IL2 to the LoRA intermediate.
- MLP PTX is emitted with CFTP FP8 weights, CFIP IF4 on the post-activation tensor that flows into the next attention.
- Backward PTX is emitted for each forward kernel, honoring the save-activation contract (none in Tier B; the backward has explicit recompute) and CFIP's dequant-on-boundary.

Stage 7 — Runtime. The compiled binary is loaded. FASE's Deferred mode runs: for each training step, the forward is computed, the loss backward runs, gradients accumulate directly into MHat/VHat, AdamW applies with bias correction via `ns1_bias_correction_inv`. No gradient buffer is allocated. Optimizer state lives in memory throughout.

This walkthrough touches every technique except the ones explicitly disabled in this example (CPDT, CPKD, CNAG, CPFT, CTQS, PCA, @export). The flow through the seven stages makes every technique's contribution visible.

§8 Adding a fifteenth technique

When a new technique is added to NSL, the composition paper must be updated. The checklist below describes the minimum updates required:

1. **Decide the stages.** Which of the seven stages does the technique operate in? If more than one, describe the hand-off contract explicitly.
2. **Write the technique's §3 entry.** Stages, what it produces, what it reads, what it assumes, downstream readers.
3. **Add cross-technique invariants.** For every contract the new technique imposes or respects, add a numbered invariant to §4. Do not renumber existing invariants; use the next available number.
4. **Add ordering constraints.** If the new technique requires a decision to happen before or after another, add an entry to §5.
5. **Enumerate failure modes.** At least three failure modes — what happens when the new technique composes with two or three existing techniques in specific adversarial configurations.
6. **Update the walkthrough in §7.** If the new technique is active in the canonical example, add its contribution.
7. **Regression-test the existing invariants.** Every existing invariant must continue to hold after the new technique is added. Run the full integration test suite.

Steps 1–6 are documentation. Step 7 is the code-side verification that the new technique does not break the compositional correctness of the existing thirteen.

A technique that cannot be described by §§3.1–3.14's format is probably not yet well-designed. The discipline of mapping onto this format has caught under-specified design twice in project history (once was the grad-context bridge initial design, which did not specify its behavior in MoE; see the connections paper for the remedy).

§9 Conclusion

The fourteen NSL technique papers describe how to implement each technique in isolation. This composition paper describes what emerges when they run together: seven pipeline stages, twenty-three cross-technique invariants, twelve decision-ordering constraints, fourteen characteristic failure modes, and an end-to-end walkthrough that shows every technique's place in the flow.

The value of this document is negative rather than positive — nothing in it is novel on its own. Its job is to prevent the next fifteenth-technique implementer from breaking invariant I-13 or decision-order O-05 or running into failure mode F-09. That prevention is worth more than the sum of the novel content.

The next two papers in this series describe, respectively, the extension directions NSL has not yet addressed (five uncovered technique domains), and the unexploited cross-paper wins plus individual paper refinements that this audit surfaced.

End of paper.